بسم الله الرحمن الرحيم

Nama : Reyhandhika Zikri Prijadi

Kelas/No,Absen : TI_1G/26

NIM : 254107020219

Prodi : D4 Teknik Informatika

Code Faktorial :

```
package BruteForceDivideConquer.minggu5;


public class Faktorial {


    int faktorialBF(int n){
        int fakto = 1;
        for(int i=1; i<=n; i++){
            fakto =fakto*i;
        }
        return fakto;
    }
    int faktorialDC(int n){
        if(n==1){
            return 1;
        }else{
            int fakto = n *faktorialDC(n-1);
            return fakto;
        }
    }
}
```

Code menggunakan MainFaktorial:

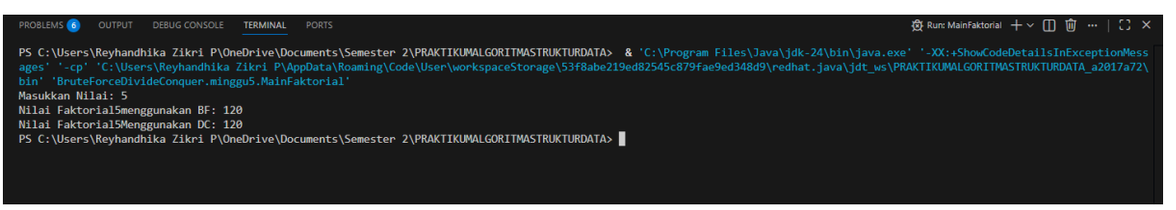
```
package BruteForceDivideConquer.minggu5;


import java.util.Scanner;


public class MainFaktorial {
    public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Masukkan Nilai: ");
      int nilai = sc.nextInt();
       Faktorial fk = new Faktorial();
    System.out.println("Nilai Faktorial" +nilai+"menggunakan BF: " +fk.faktorialBF(nilai));
    System.out.println("Nilai Faktorial" +nilai+"Menggunakan DC: "+fk.faktorialDC(nilai));


  }
}
```

Hasil :



5.2.3. Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

Jawaban: Pada method FaktorialDC() bagian If dan Else memiliki fungsi yang berbeda dalam rekursi bagian If digunakan sebagai kondisi dasar(base case) dari rekursi. Pada kode if(n==1){ return 1;}, Program akan menghentikan panggilan fungsi secara berulang Ketika nilai n sudah mencapai 1. Hal ini karena nilai factorial dari 1 adalah 1 sedangkan bagian else digunakan untuk menjalankan proses rekursif. Pada bagian ini program akan menghitung factorial dengan cara mengalihkan nilai n dengan hasil dari pemanggilan method FaktorialDC(n-1) Dengan kata lain, Method tersebut akan terus memanggil dirinya sendiri hingga mencapai kondisi dasar. Jadi If berguna sebagai penghentian rekursi, sedangkan else berfungsi untuk melanjutkan proses perhitungan factorial secara rekursif.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

Code:

```
 int faktorialBF(int n){
   int fakto = 1;
   int i = 1;

   while(i <= n){
      fakto = fakto * i;
      i++;
   }

   return fakto;
}
```

Jawaban : Perulangan While akan Terus berjalan selama nilai I masih kurang atau sama dengan n, dan setiap iterasi nilai fakto akan dikalikan dengan i.

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !

Jawaban :

fakto *= i;
Merupakan bentuk singkat dari:

fakto = fakto * i;

Operasi ini digunakan pada perulangan (iteratif) dalam metode Brute Force. Nilai fakto akan dikalikan secara bertahap dengan nilai i pada setiap iterasi.

int fakto = n * faktorialDC(n-1);

Kode ini digunakan pada rekursi dalam metode Divide and Conquer. Nilai fakto diperoleh dari hasil perkalian antara n dengan hasil pemanggilan fungsi faktorialDC(n-1).

Kesimpulannya:

- fakto *= i → proses iterasi menggunakan perulangan
- n * faktorialDC(n-1) → proses rekursif menggunakan pemanggilan fungsi

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

Jawaban : FaktorialBF() menggunakan penddekatan Brute Force, yaitu menghitung factorial dengan cara perulangan dari 1 sampai n. Nilai factorial dihitung secara berharap dengan mengalihkan setiap angka dalam loop. Sedangkan faktorialDC menggunakan pendekatan Divide and Conquer dengan Teknik rekursi. Nilai factorial dihitung dengan memecah masalah menjadi masalah lebih kecil, yaitu menghitung factorial(n-1) hingga mencapai kondisi dasar.

5.3.1. Langkah-langkah Percobaan

Code pangkat :

```
package BruteForceDivideConquer.minggu5;


public class pangkat {
     int nilai,pangkat;


   pangkat(int n, int p){
      nilai = n;
      pangkat = p ;
   }
   int pangkatBF(int a, int n){
      int hasil = 1;
      for(int i=0; i<n; i++){
         hasil = hasil*a;
      }
      return hasil;
   }
   int pangkatDC(int a, int n){
      if(n==1){
         return a;
      }else{
         if(n%2==1){
            return (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
         }else{
            return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
         }
      }
   }
}
```

Code MainPangkat:

```
package BruteForceDivideConquer.minggu5;


import java.util.Scanner;


public class Mainpangkat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumklah elemen: ");
        int elemen = sc.nextInt();


        pangkat[] png = new pangkat[elemen];
        for(int i =0;i<elemen;i++){
            System.out.print("Masukkan nilai basis elemen ke-" +(i+1)+": ");
            int basis = sc.nextInt();
            System.out.print("Masukkan nilai Pangkat elemen ke-"+(i+1)+": ");
            int pangkat = sc.nextInt();
            png[i] = new pangkat(basis, pangkat);
        }
        System.out.println("HASIL PANGKAT BRUTEFORCE: ");
        for (pangkat p : png){
            System.out.println(p.nilai+"^"+p.pangkat+": " +p.pangkatBF(p.nilai, p.pangkat));
        }
        System.out.println("HASIL PANGKAT DIVIDE AND CONQUER:");
        for (pangkat p : png){
            System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatDC(p.nilai,p.pangkat ));
        }
    }
}
```

Hasil :

```
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\Reyhandhika Zikri P\OneDrive\Documents\Semester 2\PRAKTIKUMALGORITMASTRUKTURDATA>  & 'C:\Prog
ages' '-cp' 'C:\Users\Reyhandhika Zikri P\AppData\Roaming\Code\User\workspaceStorage\53f8abe219ed82545c87
bin' 'BruteForceDivideConquer.minggu5.Mainpangkat'
Masukkan jumklah elemen: 2
Masukkan nilai basis elemen ke-1: 4
Masukkan nilai Pangkat elemen ke-1: 5
Masukkan nilai basis elemen ke-2: 6
Masukkan nilai Pangkat elemen ke-2: 7
HASIL PANGKAT BRUTEFORCE:
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
4^5: 1024
6^7: 279936
PS C:\Users\Reyhandhika Zikri P\OneDrive\Documents\Semester 2\PRAKTIKUMALGORITMASTRUKTURDATA>
```

5.3.3. Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

Jawaban : Method pangkatBF() menggunakan pendekatan Brute Force dengan cara melakukan perkalian berulang sebanyak nilai pangkatnya. Program melakukan perulangan dari 0 sampai n dan setiap iterasi nilai hasil dikalikan dengan basis. Sedangkan pangkatDC() menggunakan pendekatan Divide and Conquer dengan rekursi. Metode ini membagi perhitungan pangkat menjadi bagian yang lebih kecil menggunakan konsep:

$$a^n = (a^{n/2}) \times (a^{n/2})$$

Jika pangkat ganjil maka dikalikan lagi dengan a. Pendekatan ini lebih efisien karena jumlah operasi yang dilakukan lebih sedikit dibandingkan perulangan biasa.

2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!

Jawaban : Pada tahap Combine pada program tersebut terdapat pada pemanggilan/ rekursif dari pangkatDC()

Code :

```
if(n%2==1){
    return (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
}else{
    return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
```

3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?

Jawaban : Bisa tapi sebenarnya parameternya tidak wajib, karena nilai(nilai) dan perangkat(perangkat) sudah menjadi atribut dalam class.

4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

Jawaban: PangkatBF() menghitung nilai perangkat dengan perkalian berulang menggunakan perulangan sampai jumlah pangkat tercapai. Sedangkan pangkatDC() menghitung pangkat menggunakan rekursi dengan membagi masalah menjadi dua bagian yang lebih kecil, kemudian hasilnya digabungkan Kembali.

5.4.1 Langkah-Langkah Percobaan

Code sum :

```
package BruteForceDivideConquer.minggu5;


public class sum {
    double keuntungan[];


    sum(int el){
        keuntungan = new double[el];
    }
    double totalBF(){
        double total =0;
        for(int i = 0; i<keuntungan.length;i++){
            total = total+keuntungan[i];
        }
        return total;
    }
    double totalDC(double arr[], int l, int r){
        if(l==r){
            return arr[l];
        }
        int mid = (l+r)/2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid+1, r);
        return lsum+rsum;
    }
}
```

Code Mainsum:

```
package BruteForceDivideConquer.minggu5;


import java.util.Scanner;


public class mainsum {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Masukkan Jumlah elemen: ");
      int elemen = sc.nextInt();
      sum sm = new sum(elemen);
      for(int i=0;i<elemen;i++){
         System.out.print("Masukkan Keuntungan ke-"+(i+1)+": ");
         sm.keuntungan[i] = sc.nextDouble();
     }
     System.out.println("Total Keuntungan Menggunakan BruteForce: "+sm.totalBF());
     System.out.println("Total Keuntungan mengguna Dive and Conquer:
"+sm.totalDC(sm.keuntungan, 0, elemen-1));
   }
}
```

Hasil :

```
onMessages' '-cp' 'C:\Users\Reyhandhika Zikri P\AppData\Roaming\Code\User\workspaceStorage\53f8abe219ed
17a72\bin' 'BruteForceDivideConquer.minggu5.mainsum'
Masukkan Jumlah elemen: 5
Masukkan Keuntungan ke-1: 10
Masukkan Keuntungan ke-2: 20
Masukkan Keuntungan ke-3: 30
Masukkan Keuntungan ke-4: 40
Masukkan Keuntungan ke-5: 50
Total Keuntungan Menggunakan BruteForce: 150.0
Total Keuntungan mengguna Dive and Conquer: 150.0
PS C:\Users\Reyhandhika Zikri P\OneDrive\Documents\Semester 2\PRAKTIKUMALGORITMASTRUKTURDATA>
```

5.4.3. Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

Jawaban : Variable mid digunakan untuk membagi array menjadi dua bagian yaitu kiri dan kanan. Yang akan diproses secara rekursif

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

Jawaban: double lsum= totalDC(arr, l,mid); dan double rsum = totalDC(arr, mid+1,r); berfungsi untuk menghitung total keuntungan dari bagian kiri dan kanan terpisah melalui rekursi (tahap divide)

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

Jawaban: Penjumlahan lsum +rsum Adalah tahap combine untuk menggabungkan hasil dari kedua bagian sehingga diperoleh total keseluruhan array.

4. Apakah base case dari totalDC()?

Jawaban : Base case terjadi Ketika l==r, yaitu hanya tersisa satu elemen, maka method menggantikan nilai arr[l].

5. Tarik Kesimpulan tentang cara kerja totalDC()

Jawaban: Method totalDC() menerapkan algoritma Divide and Conquer: memecah array menjadi dua bagian secara rekursif hingga elemen tunggal, lalu menjumlahkan kembali hasil dari setiap bagian untuk mendapatkan total keseluruhan.